

Article

Real-Time Anomaly Detection of Network Traffic Based on CNN

Haitao Liu ^{1,2} and Haifeng Wang ^{3,4,*} ¹ Business School, University of Shanghai for Science and Technology, Shanghai 200093, China² Office of Information, Linyi University, Linyi 276002, China³ School of Information Science and Engineering, Linyi University, Linyi 276002, China⁴ Shandong Provincial Network Key Laboratory Linyi University Research Institute, Linyi 276002, China

* Correspondence: wanghaifeng@lyu.edu.cn; Tel.: +86-135-6391-2296

Abstract: Network traffic anomaly detection mainly detects and analyzes abnormal traffic by extracting the statistical features of network traffic. It is necessary to fully understand the concept of symmetry in anomaly detection and anomaly mitigation. However, the original information on network traffic is easily lost, and the adjustment of dynamic network configuration becomes gradually complicated. To solve this problem, we designed and realized a new online anomaly detection system based on software defined networks. The system uses the convolutional neural network to directly extract the original features of the network flow for analysis, which can realize online real-time packet extraction and detection. It utilizes SDN to flexibly adapt to changes in the network, allowing for a zero-configuration anomaly detection system. The packet filter of the anomaly detection system is used to automatically implement mitigation strategies to achieve online real-time mitigation of abnormal traffic. The experimental results show that the proposed method is more accurate and can warn the network manager in time that security measures can be taken, which fully demonstrates that the method can effectively detect abnormal traffic problems and improve the security performance of edge clustering networks.

Keywords: software defined networks; convolutional neural networks; edge clusters; anomaly detection; anomaly mitigation

**Citation:** Liu, H.; Wang, H.Real-Time Anomaly Detection of
Network Traffic Based on CNN.*Symmetry* **2023**, *15*, 1205. <https://doi.org/10.3390/sym15061205>Academic Editors: Zhixun Su and
Silvio Pardi

Received: 26 February 2023

Revised: 23 May 2023

Accepted: 2 June 2023

Published: 4 June 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Driven by the pervasive computing applications realized by the Internet of Things, a large number of intelligent terminal devices are connected to the Internet. The massive data generated by users needs to be processed. However, the cloud computing model with centralized computing power can hardly meet the performance requirements. The user-side data are transmitted to the data center for processing, which consumes a lot of WAN bandwidth and has high network latency. The edge computing model becomes a new distributed data processing method, which responds to the user's computation request in time and reduces the network latency of data transmission by being close to the data side. With the popularity of edge computing applications, data security in edge clusters has become an important challenge. In edge computing, the harm of anomaly attacks is symmetric. The cost of anomaly detection and the execution time of anomaly mitigation tasks are asymmetric.

The network of edge computing is subjected to various types of abnormal traffic attacks such as replay, intermediary, simulation, password guessing, routing attacks, and other security attacks as well as “denial of service attacks” [1]. When an edge cluster is attacked by abnormal traffic, the edge nodes are unable to get the correct information. Then, network services are blocked, and network latency and packet loss increase accordingly. Network reliability and throughput decrease dramatically [2–4]. So, it is important to detect abnormal attacks on edge computing. Traditional network anomaly detection is simple and

easy to use. However, there are two limitations: (i) the network environment of the edge computing cluster changes dynamically, and the traditional network anomaly detection system has poor scalability and is difficult to adapt to the uncertain adjustment of the edge network; (ii) the traditional network anomaly detection is not intelligent and cannot solve the detection problem of real-time data streams in the edge computing scenario [2]. The problems of complex management and poor scalability of detection systems in edge networks are addressed by software defined networks (SDN) [5,6]. The fast anomaly identification scheme for real-time data streams is designed using convolutional neural networks (CNN).

The main contributions of this paper are as follows: (i) design a network anomaly detection system for edge-clustered SDNs to enhance the flexibility and intelligence of the anomaly detection system through global management optimization of software defined networks; (ii) design a model to address real-time data stream anomaly detection using the fusion of convolutional neural networks and software defined networks to improve the accuracy of data stream anomaly detection.

The structure of this work is as follows: Section 2 addresses related work; Section 3 on SDN introduces anomaly detection and mitigation system design; Section 4 discusses CNN-based anomaly detection model; Section 5 provides the experimental design; Section 6 discusses experimental results and analysis; and Section 7 offers a conclusion.

2. Related Work

The network anomaly flow attack initiates a large amount of abnormal traffic to the server or network through a single computing node, which causes the degradation of computer system performance or even paralysis. The traditional abnormal traffic detection method extracts the traffic features from the exchange board flow table and then implements the feature detection of abnormal flows. However, when the system receives large-scale data streams, the features of network connections, status tables and packet tags are too expensive to maintain [7]. The traditional abnormal flow detection methods extract character or numeric features from traffic packets to classify normal or abnormal traffic. Therefore, dynamic detection cannot be achieved, especially in edge computing environments where additional computational resources are still required to complete the detection [8]. Zhou extracted 256 features from traffic to build the detection model, which is only applicable to classify the traffic with large-scale data [9]; J. Kim used an LSTM neural network model to complete the classification five times on a KDD dataset [10]; Shukla designed a recurrent neural network structure to detect network traffic, which not only maintains the network forwarding performance but also obtains better detection accuracy. However, the method is subject to performance degradation after the virtualization of computing resources [11]. Zavrak proposes a scheme for modular prevention and detection of network attacks. However, it lacks analysis of the impact of important parameters on the controller load, so the accuracy of anomaly detection is not high [12]; Dan Tang proposes an adaptive Kohonen network model to achieve fine-grained anomaly flow detection for low-rate denial-of-service LDoS attacks, but the performance loss is high, and the detection accuracy is low [13]. Zhang proposed a network intrusion detection method based on a deep hierarchical network and raw traffic data; however, a traffic collection system was not designed to collect real traffic data for analysis, and no in-depth improvement of the hierarchical network model was made to detect unknown types of attacks [14]. Lopez-Martin et al. propose a new network intrusion detection method based on conditional variational autoencoder suitable for IoT networks by integrating the intrusion labels inside the decoder layers [15]. The edge computing completes data processing close to the data side, with a higher degree of virtualization of computing resources, and the uncertainty of traffic features in edge networks is more complex [16]. Therefore, traditional abnormal flow detection methods are challenged to handle the network abnormal flow detection work in edge computing.

On the other hand, the software defined network is an important solution to address the network management of edge computing. SDN achieves zero configuration and dynamic adjustment of network management. SDN is used in network anomaly flow detection to extract packets online and achieve online real-time detection of packets. Jaraweh proposes a software defined network architecture model to realize the basic functions in edge computing and reduce the management cost of network devices in edge clusters [4]. Ali analyzed and compared the detection methods of SDN-based machine learning and deep learning in response to the distributed denial-of-service attack DDoS; however, there is a lack of experiments on detection accuracy in real networks [17]. Some gateway traffic can be tightly controlled due to the SDN capabilities of the local edge cloud. The “softwareisation” of a classically hardware-driven business built around routers and servers where services are deployed will result in cheaper and more agile operations [18]. McKeown optimized the design of SDN controllers to improve the global control of network resources and address the limitation of network edge traffic in edge clusters [19]. Lin et al. proposed software defined intelligent edge architecture to support the construction of various distributed network working services and applications [20]. In response to DDoS attacks on edge devices, Ren designed a software defined network anomaly detection framework to enhance intelligent detection [21]. Zhou designed the CoWatch framework to detect and discard attack flows of the distributed SDN [22]. Yang modified the service of OpenFlow switch in SDN to add anomaly flow detection and mitigation to the service to handle simple distributed denial-of-service attacks. However, there is a lack of performance tests for complex denial-of-service attacks and a lack of full utilization of the global sensing capabilities of SDN to handle the detection task [23]. Harun used the feedforward convolutional neural networks to design detection of LDoS attacks in IoT SDN, while lacking performance loss analysis in real-time systems [24].

In summary, the traditional anomaly flow detection model uses earlier attack datasets and cannot verify the effectiveness of current new attacks, making it difficult to handle the anomaly flow detection work in the edge computing environment. In addition, the current anomaly stream detection model belongs to a shallow classification model. It can achieve a better detection effect when the data stream feature dimension is small; when the data stream size is large and the feature dimension is high, the detection and classification effect is poor, and the intelligence of the anomaly detection model needs to be enhanced to handle the real-time detection demand in edge computing.

3. SDN Anomaly Detection and Mitigation System Design

The SDN and CNN are used in edge networks to learn their temporal and spatial features from the original network data, preserving as much relevant information about the traffic as possible. The CNN model does not require any prior knowledge to extract traffic features, can automatically extract traffic features with specific meanings from the input data for training, and can discover the main features that have a significant impact on abnormal traffic detection. The system consists of five main modules: data processing, model building, model training, real-time detection, and anomaly mitigation. The system prototype runs under Ubuntu and uses CNN to train the acquired data and save the trained model. The basic SDN network is built using Mininet, the data plane is built using OVS, and the controller uses RYU. After starting RYU, the whole network starts running and the controller will detect the data flow in the network in real time. Because it is an inspection of the edge network, the OVS exchange board of the edge network does not add the flow table for forwarding packets to the other exchange board. The OVS exchange board sends the processing result of unknown packets to the controller, and all packets flowing through it can be detected. When the packet-in packet enters, the packet can be received in the controller. The packets are decomposed, the required feature information is extracted, and the list of extracted features is transmitted to the trained model, and then recognition judgments are made to give the output. The RYU controller is responsible for managing and creating traffic rules that have matching fields and instructions associated with the

traffic in its traffic table. These rules are sent to the OVS exchange board, which then adds the entries to its own traffic table. The normal traffic is added to the traffic table for normal transmission scheduling, while the abnormal traffic is identified as attack packets and mitigation policies are implemented. The knowledge gained from the anomaly status, extracted features, IP, and port addresses is used to identify emerging anomalies.

The important measures for applying these policies are as follows:

- (1) Blocking data flows from the specific IP or advertised port address.
- (2) Abandoning packets that are forwarded in a specific traffic flow.
- (3) Balancing the load by redirecting attacks to other idle servers [5].
- (4) Finally, implementing a GUI interface to display the real-time data monitoring situation.

The SDN abstracts the control flow management functions from the forwarding nodes (FEs) and consolidates them into a logical centralized controller to obtain a unified global view of the network. The SDN controller manages all FE functions by facilitating the programming of standardized network devices, such as the exchange board, routers, gateways, or any access node. The communication architecture of SDN follows the OpenFlow protocol, enabling the controller to perform flow-level control. The SDN framework consists of three decoupled planes: the data plane, the control plane, and the application plane [25].

3.1. Architecture

Combining the artificial intelligence method to study the SDN, the knowledge defined network (KDN) architecture is proposed, aiming to obtain network information through the advantage of centralized control of SDN and solve complex network control and management problems using intelligent decision making. The framework is shown in Figure 1. The KDN mainly consists of a data plane, control plane, and decision management plane, aiming to provide automated network control and management for SDN through the closed-loop control. The analysis platform in the control plane monitors the fine-grained traffic information in the data plane in real time and queries the control and management status of the SDN controller to provide a global view, define the network topology, collect and process information provided by network devices, monitor and analyze the network, and ensure the long-term proper configuration and operation of the network. The decision management plane is the core of the KDN. It uses the network view information provided by the control plane analysis platform to learn and generate useful knowledge from the network and is responsible for the generation of dynamic control policies. The data plane consists of several network elements, each of which could contain one or more SDN data paths. Each SDN data path is a logical network device containing three parts: the control data plane interface agent, the forwarding engine table and the processing function, which has no control capability and needs to be controlled by the control plane to forward and process data.

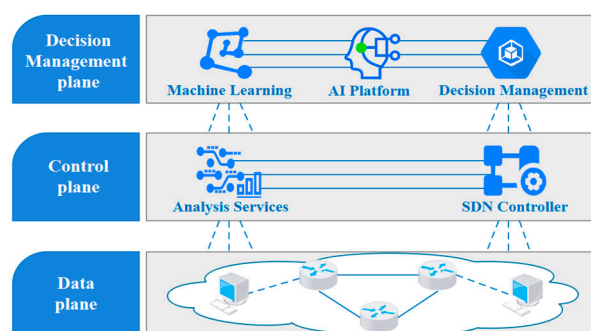


Figure 1. Framework of knowledge defined networking (KDN).

3.2. Main Functions

The main function of this work is to collect current traffic data and pass the data to the controller for feature extraction. The extracted features are pre-processed and passed to the neural network model. Then, the prediction and identification results and mitigation strategies are given. The framework is shown in Figure 2 below. Here the experiment is done using Mininet. Alternatively, multiple virtual machines can be used. The client as the sender sends data to the edge service cluster as the receiver through the edge cluster network. The CNN server and SDN controller in the edge cluster network predict the availability of data in real time. Then, they send different types of data according to the corresponding policies to further improve the security and availability.

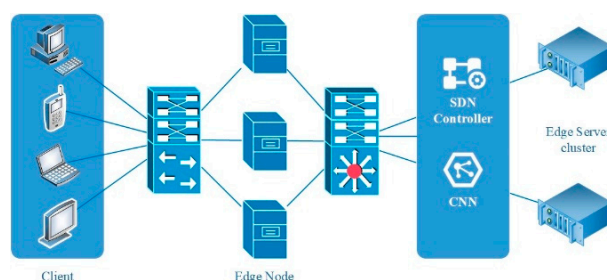


Figure 2. Real time anomaly detection architecture of edge cluster network based on SDN and CNN.

3.3. Implementation Principle

When the controller receives an unknown packet via the OpenFlow protocol, the packet contains complete information about the packet. After the controller receives the packet, it will parse the packet's encapsulation. That is, it parses each layer of the packet, extracting the information from each layer of the packet layer by layer like peeling an onion. In this way, a flow table can be created for the packet based on the extracted information. Then, the packet-out packet is passed to the exchange board through the OpenFlow protocol. The packet-out packet contains the interface to forward the packet. Based on this principle, the packets submitted by the exchange board can be processed, and then the required information can be extracted and passed to a trained model for identification and prediction, the results of which are shown in Figure 3. Firstly starts the Mininet and RYU controller. And sends packets at this time the switch forwards the first packet to the controller. The controller transmits the packet to the CNN anomaly detection model. After data preprocessing, CNN training and modeling are used to identify the abnormal traffic. If it is normal traffic, the packet is directly transmitted. If abnormal traffic is identified, blocking, redirect, and discard are three ways to mitigate the attack of anomalous traffic.

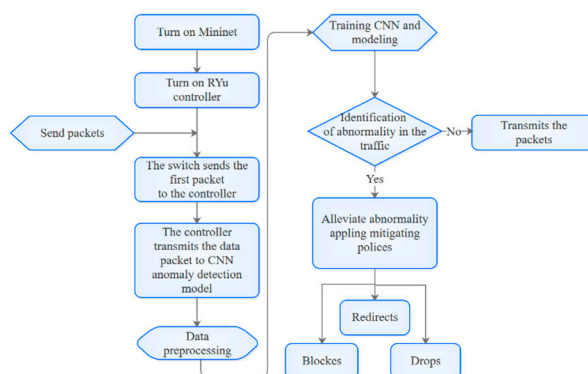


Figure 3. Implementation schematic diagram.

Abnormal traffic can damage the normal operation of the network. The recovery strategy focuses on the management and reconfiguration of detection and recovery mecha-

nisms that function as autonomous components in network [26]. The system implements procedures to locate the abnormal traffic and the affected devices and can mitigate these threats by reconfiguring the policy [27]. The main operations of implementing policies include forwarding, dropping, or modifying packet fields. Table 1 lists the implemented policies. Blocking cuts off the incoming traffic from a malicious host (port scanning attack) while blocking communication from a specific host to a specific service in the target IP address. Dropping drops protocol data units at a specific address. Redirecting distributes the load of network traffic generated by the abnormal attacks to multiple servers to avoid degrading service performance.

Table 1. Mitigation policies and actions taken.

Mitigation Policies	Action Taken
Blocking	Block all the traffic flow from a particular address
Dropping	Drop the PDU for the particular address
Redirecting	Directs the traffic flow to the utilization of fewer servers

3.4. Implementation Plan

The prototype system starts an Ubuntu 16.04 virtual machine on the computer, starts Mininet in a VMware virtual machine, and sets up a simple network for testing, as shown in Figure 4, which depicts the working principle of SDN. The RYU controller is the controller of SDN, and s1, s2, and s3 are the switches.

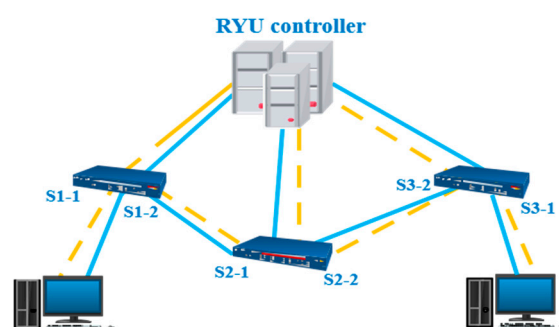


Figure 4. Mininet network topology.

The SDN architecture is mainly composed of the SDN controller, data plane devices and applications. Its core idea is to separate the control plane and data plane. The SDN architecture can be divided into three layers in terms of central control and management of the data plane through the SDN controller.

(1) Application layer: The application layer mainly refers to the upper layer of the SDN controller applications, such as network management, network security, and other applications. The developers can use these applications through a specific SDN application program interface (API).

(2) Control layer: control layer mainly refers to the SDN controller control software. The role is to control the entire network state and topology information. Providing the control and management in the data surface, this network can quickly adapt to the variations.

(3) Data layer: The data layer mainly refers to the SDN network ports, switches, routers, gateways, and other devices whose role is to establish communication between the network nodes through the data level interconnection.

The control layer sends control instructions to the data layer through protocols, such as OpenFlow. The data layer operates based on these instructions to achieve centralized control and management of the network.

The edge cluster packets are collected and sent to the network, which receives the packets and forwards them to the controller. After the controller receives the data packets, it extracts the corresponding information according to the features and saves them.

After data pre-processing, data are transmitted to the pre-trained CNN model, and the prediction results are obtained by the model. The implementation is shown in Figure 5 and the program flow is shown in Figure 6. The impact of abnormal traffic is mitigated by mitigation strategies.

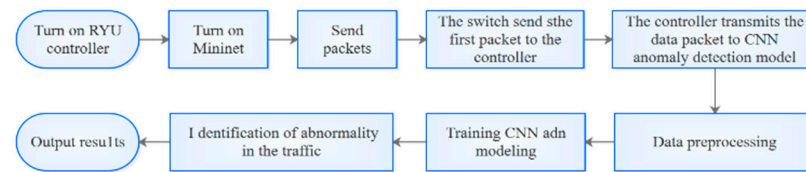


Figure 5. Implementation.

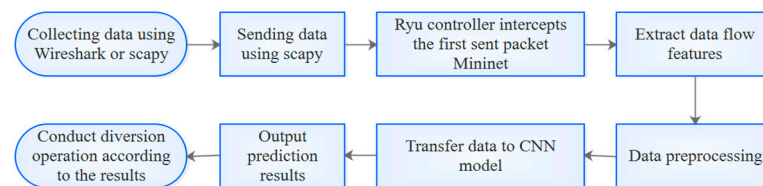


Figure 6. Program flow chart.

4. Anomaly Detection Model Based on CNN

The convolutional computation of CNN has good spatial perception ability, which has been widely used in image processing, such as facial recognition, with good results. In networks, the service packets generated by users are segmented during transmission [28]. The IP field of each service packet represents the spatial features of the traffic. Considering the spatio-temporal features of traffic data, this work uses the CNN model to extract the spatio-temporal features of traffic packets [14].

The neural network is a linear transformation when the input vector dimension is higher than the output vector. The neural network is equivalent to an encoder, which realizes the low-dimensional feature extraction of high-dimensional features. On the contrary, when the input vector dimension is smaller than the output dimension, the neural network is equivalent to a decoder, which realizes the high-dimensional reconstruction of low-dimensional features.

In mathematics, $(f * g)(n)$ is the convolution of f and g , which is defined in continuous space as:

$$(f * g)(n) = \int_{-\infty}^{+\infty} f(\tau)g(n - \tau)d\tau \quad (1)$$

The discrete definition is:

$$(f * g)(n) = \sum_{\tau=-\infty}^{+\infty} f(\tau)g(n - \tau) \quad (2)$$

A convolutional neural network is essentially an input-to-output mapping that is capable of learning a large number of mapping relationships between inputs and outputs. As long as the convolutional network is trained with known patterns, the network has the ability to map between input-output pairs in the network, user-generated traffic packets are segmented during transmission [14], and the IP field of each traffic packet represents the spatial characteristics of the traffic. We use a CNN model to extract the spatial characteristics of the traffic packets.

A convolutional neural network consists of 5 main layers as follows:

- (1) Data input layer
- (2) Convolutional computation layer
- (3) ReLU excitation layer
- (4) Pooling layer

(5) Fully connected layer

Convolution is the extraction of features from the original input. The extraction process is divided into regions. The features are extracted piece by piece. According to the increment of convolution layers, the extracted features are changed from the basic points, lines, and surfaces to the specific features that can be used to separate things.

The convolution operation uses a convolution kernel ω of size $f * f$ to perform a sliding convolution on a matrix of size $n * n$, with each sliding convolution producing a new feature. Suppose X is the input to the convolution, b is the bias term, c_i is the new feature generated by the convolution at layer i , and σ_r is the activation function ReLU [14]. Then, the new feature obtained by the convolution operation is:

$$c_i = \sigma_r (\omega * X_i + b_i) \quad (3)$$

After the convolution operation, the feature matrix of $n * n$ will generate a feature matrix of size $c = (n - f + 1) * (n - f + 1)$ that slides through a convolution kernel window of size $f * f$. After convolution, the maximum set is performed on the feature matrix c , and the maximum value in the selected window is used as the final feature [14]. The final feature matrix size is $[(n - f + 1) * (n - f + 1)]/2$.

4.1. Anomaly Detection Process

When the network is operating safely, the features of each dimension of the network are relatively stable, but when anomalies occur, there are large fluctuations. When the value of some dimensions exceeds a threshold, it is determined that an anomaly has occurred. There are many types of anomalies, and the different anomalies underlying the CNN anomaly flow detection model have different features. As shown in Figure 7, the specific process of detection is divided into two main stages, Stage1 and Stage2: Stage1 is pre-processing, and the network traffic data are collected using the time window method. The common attack takes a time window of 2 s and collects data once; the slow attack takes a time window of 5s and collects data once [29]; Stage2 is CNN training modeling with real-time anomaly detection and alerting of online data [30].

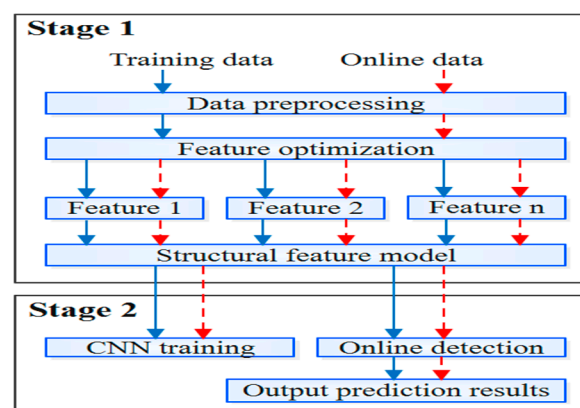


Figure 7. Abnormal flow detection model.

4.2. Anomaly Identification and Mitigation

The expected properties of the traffic are used to identify normal and abnormal traffic. If the traffic is normal, the packets will be forwarded. If the traffic contains any anomalies, a mitigation policy will be applied. The mitigation policy for abnormal traffic can be obtained by applying blocking or dropping or redirecting the traffic.

4.3. CNN Design

The convolutional neural network generally consists of a multilayer structure with an alternating combination of input, output, convolutional, and fully connection layers [30–32].

Each layer consists of multiple neurons, and each neuron represents a feature of the data. As the number of layers of the convolutional neural network increases, the training parameters also increase, while the weight sharing strategy of the convolutional layers greatly alleviates the problem of parameter explosion.

Assuming that the input is $\{x_1x_2, \dots, x_9\}$, the activation function is $f(x)$, and the output is $y = f(x_1w_1 + x_2w_2 + x_3w_3 + \dots + x_9w_9 + b)$, and a feature table is obtained for each convolution operation. Each convolution operation uses the same convolution kernel and shares weight parameters, so that the number of weight parameters is greatly reduced.

(1) Overall structure

The features of the network data stream are extracted, and 16 features form a $4 \times 4 \times 1$ matrix, where “1” represents the number of colored channels. These $4 \times 4 \times 1$ matrices are sent into the CNN shown in Figure 8. The CNN shown in the figure includes an input layer, a convolution layer, a full connection layer, and an output layer, which also shows two key operations of the activation function and dropout.

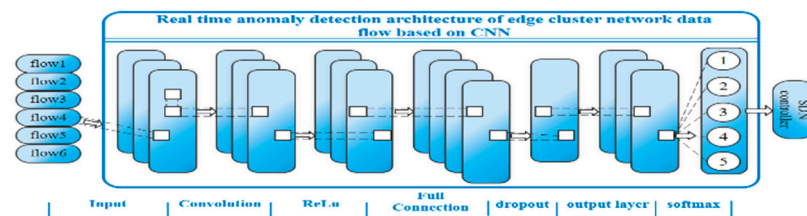


Figure 8. CNN structure.

(2) Convolutional layer

The size of the convolutional kernel in the convolutional layer of the model is denoted as $kwidth \times kheight \times kchannels$, where $kwidth$ and $kheight$ denote the width and height of the convolutional kernel respectively; $kchannels$ denotes the number of channels of the convolutional kernel; the convolutional kernel in the convolutional layer is $3 \times 3 \times 1$; stride is 1, padding is 1, and there are 32 convolutional kernels in total. Stride refers to the distance that the convolutional kernel moves each time. Assuming that the size of the input data is $w_1 \times h_1 \times d_1$, the size of the output features is $w_2 \times h_2 \times d_2$. Based on the size of the input data, the size of the output features and the size of the convolution kernel, the formula for the size of the output features can be deduced as follows:

$$w_2 = (w_1 - kwidth + padding \times 2) / stride + 1 \quad (4)$$

$$h_2 = (h_1 - kheight + padding \times 2) / stride + 1 \quad (5)$$

$$d_2 = d_1 \quad (6)$$

Passing through the convolution layer, each $4 \times 4 \times 1$ input stream produces $4 \times 4 \times 32$ features. Each convolution kernel in the formula must be output by the activation function $f(x)$ in the operation.

The convolution operation is essentially a linear operation, while the activation function is a nonlinear function that allows better generalization of the neural network. There are many commonly used activation functions, and the ReLU function is selected because of the gradient decay easily caused by the Sigmoid function during back propagation. The derivative of the ReLU function is 0 at $x < 0$. However, when $x \geq 0$, the derivative of the ReLU function is 1. This can completely transfer the gradient of y to x without causing the gradient to disappear.

(3) Full connection layer

The full connection layer is to connect all the neurons output from the previous layer to the neurons in the current layer. The convolutional neural network in this paper has

two full connection layers. However, when each operation occurs, not all the weights are involved in the operation, only some neurons connected by the weights are involved in the operation under the dropout [33]. The obtained results go to the output layer.

(4) Dropout

In order to prevent model overfitting, a dropout operation is added after the output of the full connection layer when designing the convolutional neural network structure. A simple “multiply by zero” algorithm is used in this paper. During the training, the excitation function of the neuron in the full connection layer where the dropout is located is set to zero with a probability of 0.55.

(5) Output layer

The output layer has 23 neurons, corresponding to 23 anomalies. They are responsible for receiving the Softmax function classifier used as the output of the probability values of the samples to which they belong [34].

5. Experiment Design

Here an anomaly detection network model consisting of a convolutional neural network algorithm is designed to automatically extract traffic features. The software and hardware configuration information of the host system is shown in Table 2.

Table 2. Host hardware and software configuration information.

Name	Version
CPU	Intel (R) Xeon (R) E31230 V2@3.3Ghz
Memory	DDR3 1600 Mhz (16 G)
OS	Ubuntu 16.04.7 LTS
RYU	4.32
Development Platform	pycharm-community-2018.2.8
Mininet	2.2.0
python	3.5
tensorflow	2.3.1
scapy	2.4.4
numpy	1.18.5
Keras	2.4.3
Pandas	0.24.2
matplotlib	3.0.3

5.1. Data Set

The original traffic packets are used as the analysis object for network anomaly detection, which does not require filtering and design for the traffic features to be extracted. Compared with the commonly used manual traffic packet data extraction methods, this method can retain all feature information of each traffic packet. In Wireshark, in this paper, it can be seen that the original traffic packets are some hexadecimal codes, as shown in Figure 9.

The process of extracting traffic characteristics from the original traffic packet data is as follows:

Data: each traffic packet has an Ethernet layer, a network layer, a transport layer, and an application layer. This document does not use the data from the Ethernet layer and the version and differentiated service fields from the network layer. Anderson et al. analyzed the traffic features [35] because the three fields at the Ethernet layer are the MAC source address, MAC destination address, and protocol. These fields are not usually used as features of traffic packets.

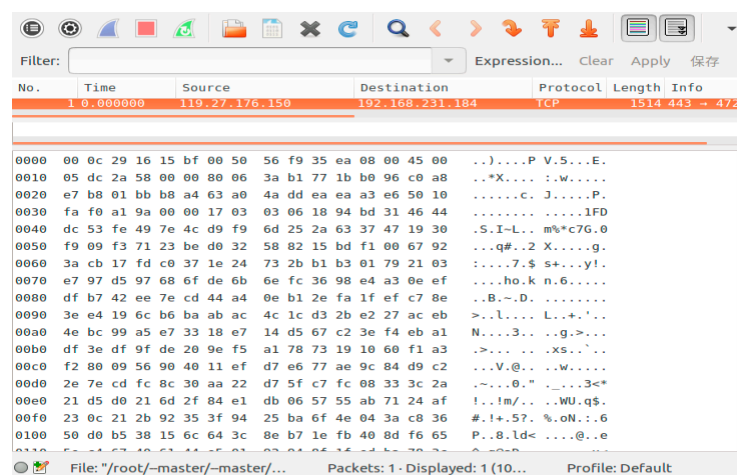


Figure 9. Wireshark information for raw traffic packets.

As elaborated by Weller Fahy, a key problem of most intrusion detection datasets is the lack of a sufficient number and type of traffic packages [36]. The experience is conducted using recently released datasets containing more flows and types, which are more reliable datasets for validation and testing than others. The CICIDS2017 dataset is an intrusion detection and intrusion prevention dataset from the Canadian Institute for Cybersecurity Research, opened in 2017 [37,38], which is used in training and testing data for CNN models. In this paper, the generated data streams are identified according to the CICIDS2017 data labeling method to obtain realistic and reliable labels. The 200,000 connection records in the CICIDS2017 dataset are imported into t-SNE [39], and their visual representation is given in Figure 10. T-distributed stochastic neighbor embedding (tSNE) is a common nonlinear dimensionality reduction method, which is very suitable for visualizing high-dimensional data by reducing them to 2 or 3 dimensions. The positions of the x and y coordinates indicate the similarity of the data samples, and in general similar data points will be clustered together, while data points of different categories will form different clusters.

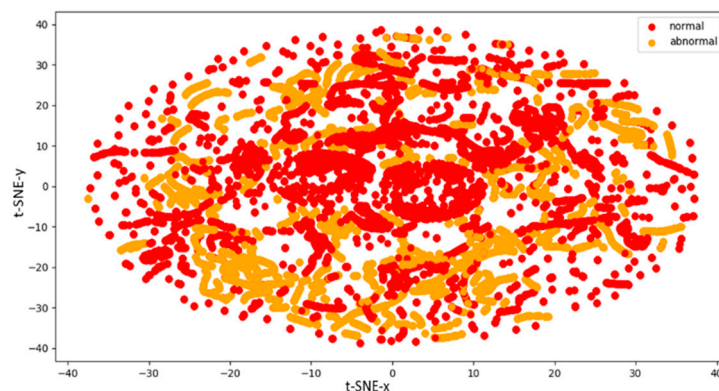


Figure 10. Visual representation of CICIDS2017.

The CICIDS2017 dataset has been recently released and contains recent attacks; almost all attacks are nonlinearly separable, and the connection records are considered to be more complex. In addition, the CICIDS2017 dataset is characterized by real-time network traffic [14,27,38].

5.2. Data Pre-Processing

The steps of data pre-processing are as follows:

- (1) Remove feature lines
- (2) Convert text to numeric values

- (3) Remove the line where the missing value is located
- (4) Remove the labeled columns to get the feature set for training
- (5) Normalize feature data and perform unique hot coding on label columns

After obtaining the data packets in the controller, the corresponding data can be extracted based on the features. In this paper, the following features are used: source IP, source port, destination IP, destination port, protocol, timestamp, total Fwd packets, minimum packet length, maximum packet length, average packet length, FIN flag, SYN flag, RST flag, PUSH flag, ACK flag, and URG flag. Finally, if there is only one packet, then Total Fwd Packets, Min Packet Length, Max Packet Length, and Packet Length Mean are all the same.

According to the layered display, the IP layer extracts the source IP, destination IP, and the protocol used by the upper layer. The TCP layer extracts the source port, destination port, fin, syn, rest, push, ack, urg, and other flag information. The UDP layer extracts only the source and destination ports.

Finally, the packet size, and the maximum, minimum, and average size of the packet are calculated.

5.3. Simple Switch Design

Since Ryu has its own Simple Switch, it only needs to inherit this class when implementing the controller. But the mac of the captured code is basically the same thanks to how the code learns the mac itself. So, when the first packet is sent to the Mininet network, the switch has already learned the mac. When the second packet is sent to the Mininet network, the controller cannot receive any packet because the packet has already been forwarded through the switch's local flow table. To solve this problem, we designed another version of Simple Switch.

This Simple Switch will not have the ability to pass flow tables (using flooding), while other switches have the ability to forward flow tables. Although this will increase the burden on the edge nodes, it can create a packet detection defense wall on the edge network, which is well worth it.

5.4. Experiment Setup

The experiments are designed and conducted in the Python programming language and all the proposed methods use Keras and Tensor flow backend libraries.

Since the data are in text form, the one-dimensional convolution of CNN is used to process the data according to the formula $(N + 2P - F)/S + 1$.

Where N refers to the latitude size of the input data, and here it refers to the size of the features. There are 16 features in this paper.

P refers to the size of the fill data, F is the size of the convolution kernel, which is 0 by default, and S is the step size, which is 1 by default.

For example, there is a network structure that can be: the input data are 16,161, at this time the N is 16.

The first layer of convolution: F is 2,32 filters, S is 2, P is 0, then the output size is 8832.

The second layer of convolution: F is 2,64 filters, S is 2, and P is 0, then the size of the output is 4464.

The third layer of convolution: F is 2,128 filters, S is 2, and P is 0, then the output size is 22,128.

The fourth layer of convolution: F is 2,512 filters, S is 2, and P is 0, then the output size is 11,512.

At this point, after sufficient connection, the final result can be output to quickly build an efficient processing model.

The model uses two layers, each of which includes a convolutional layer and a pooling layer. Finally, it goes through an average pooling layer and a fully connected layer. The adjustment of hyper-parameters plays a key role in determining the performance of the CNN network model. Several parameters, including the number of convolutional layers,

the number of filters, the size of the filters, the step size, the padding method, and the batch size, are important to obtain the highest performance of the CNN model. In practice, there is no magic rule for choosing the optimal value of hyper-parameters; that is, many experiments and trials are the suggested way to find the best model structure (Kim et al., 2020). Therefore, during the implementation, several cases and combinations were tested to find the best hyper-parameters. Table 2 shows the hyper-parameters considered for the CNN model in this text.

The number of CNN convolutional layers depends on the features of the input traffic matrix. Therefore, we used two convolutional layers with output sizes of 32 and 64. Each layer uses a filter of size 3×3 with a span equal to 1. The second convolutional layer is followed by another maximum pooling layer of size 2×2 with a span equal to 1. The pooling layer minimizes the size of the feature matrix because each convolutional layer learns the feature representation of the previous output. For a fully connected layer with 128 neurons, the output of the pooling layer is flattened and reshaped. The nonlinear mapping function Relue is used for all layers before the output layer. Finally, the classification layer is used to classify the incoming traffic into normal and attack categories. We implement Table 3 to obtain the optimal hyper-parameters for the CNN model.

Table 3. Model parameter settings.

Hyper-Parameters	Best Value
convolutional layer	2
number of filters	32, 64
kernel size	3×3
stride	1
pooling layer	Highest (2×2)
filling method	identical
nonlinear function	reconsider
batch size	128
Number of iterations	100

The performance of the different classification methods is compared through a series of experiments and analyses. After the model is trained, test data are used to evaluate the model and show how it works with unseen data.

5.5. Anomaly Mitigation

The timing of anomaly mitigation is determined by the number of flows inserted into the exchange board traffic table to comply with the policy. Depending on the mitigation policy, traffic is either blocked, dropped, or redirected. It will take longer to redirect traffic because of the need to determine which server requests will be forwarded and which alternate server traffic will be diverted.

6. Experimental Analysis

This section evaluates the performance of the proposed model through a series of experiments on the CICIDS2017 dataset. The analysis of results is as follows.

6.1. Detection and Validation

Here the model is evaluated using the Accuracy (A), Precision (P), Recall (R), and F1 score (F). These indexes are defined as follows:

Accuracy: the number of correctly classified samples as a percentage of the total number of samples.

$$A = (TP + TN)/N \quad (7)$$

Precision: the proportion of correctly predicted positive data to predicted positive data.

$$P = TP/(TP + FP) \quad (8)$$

Recall rate (Recall): the proportion of predicted correct data to the actual positive data.

$$R = TP / (TP + FN) \quad (9)$$

F1 score (F1 score): the summed average.

$$F = 2 / (1/P + 1/R) = 2 * P * R / (P + R) \quad (10)$$

These parameters are defined as follows: TP represents the number of attack samples classified as attack class. FP represents the number of normal samples misclassified as attack class. FN represents the number of attack samples misclassified as normal class. TN represents the number of normal samples classified as normal class. Therefore, the model performance is better when A and P are larger [9,14].

The CNN-SDN model greatly improves the detection rate and unknown attack detection rate. In addition, it uses mitigation strategies to block, drop, and redirect traffic in order to reduce the severity of attacks with good expected results.

6.2. Program Operation

The screenshot of the run result is shown in Figure 11.

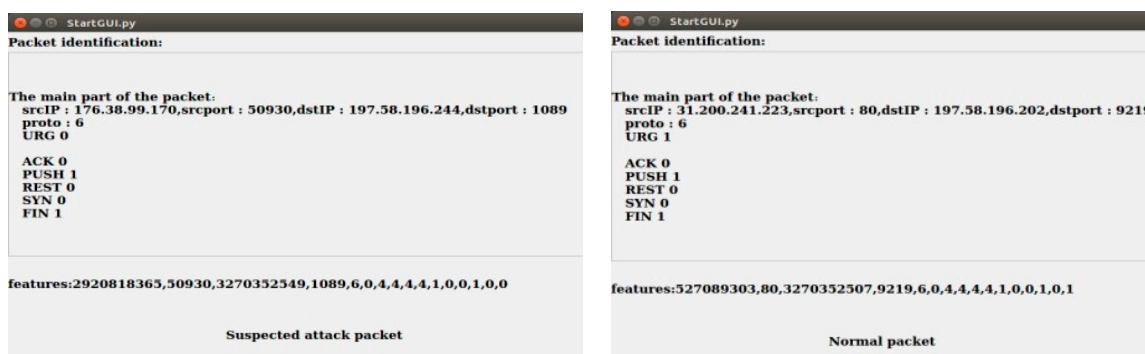


Figure 11. Running screenshot.

6.3. Result Display

- (1) Training accuracy displayed under Ubuntu (number of training times is 6).
- (2) Model training and test.

To improve the model generalization ability, the dropout technique was added to deal with the overfitting problem. In the experiments, comparison was made with and without the dropout technique, as shown in Figures 12 and 13. During the model design and testing, the dropout was set to 0.55. The final loss and accuracy of the statistical model tended to be stable.

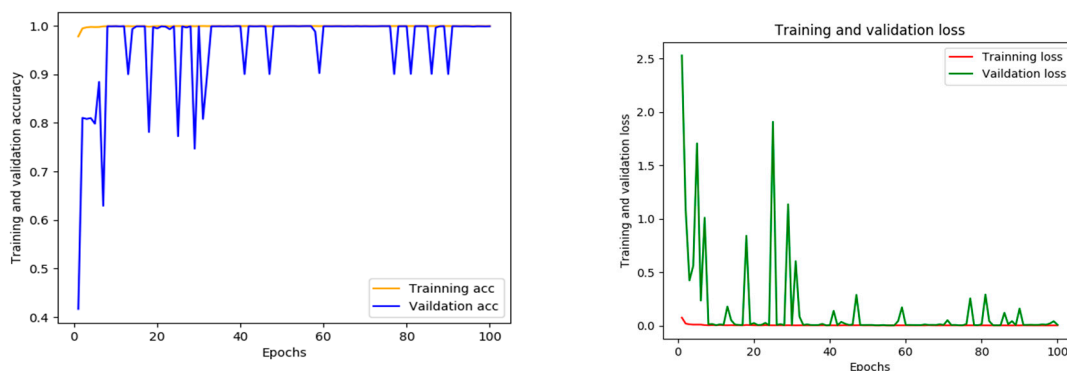
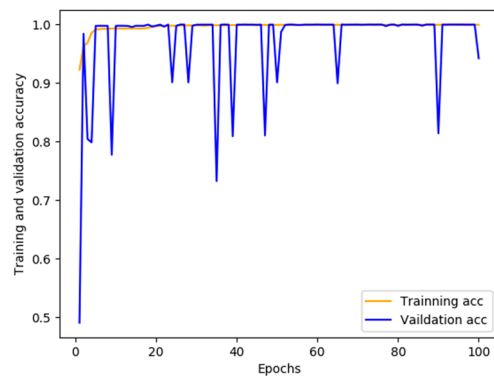
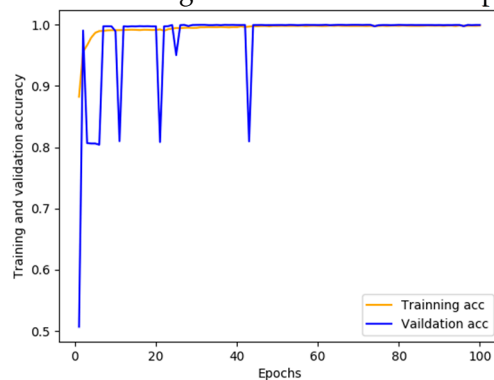
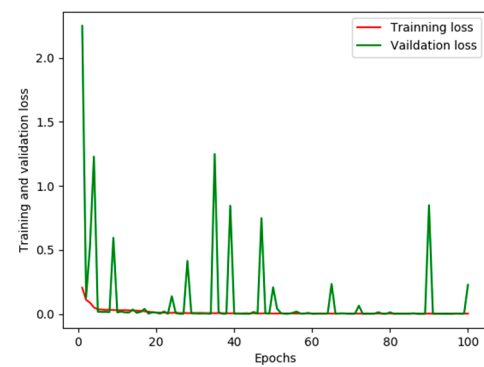


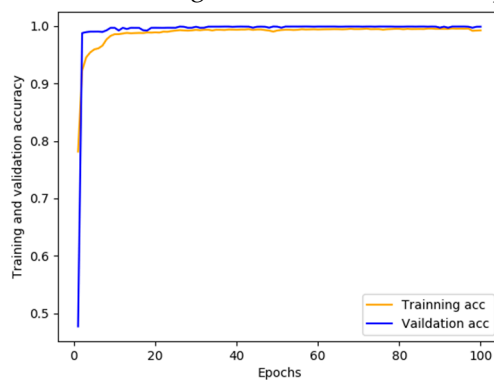
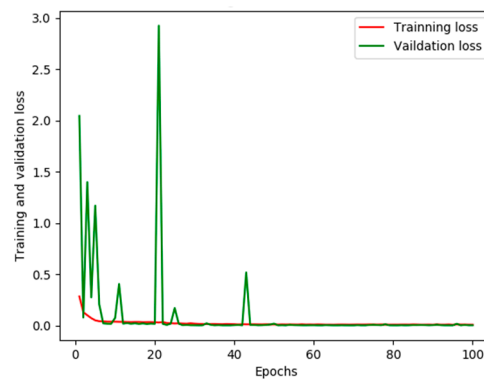
Figure 12. Trends in training and validation without dropout.



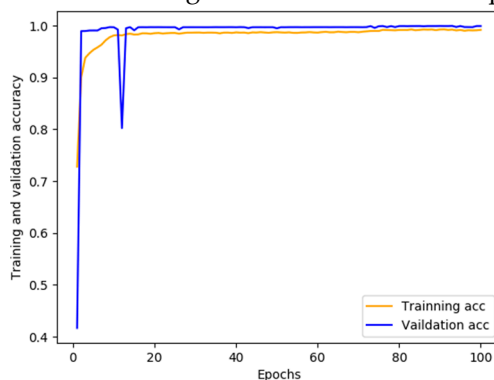
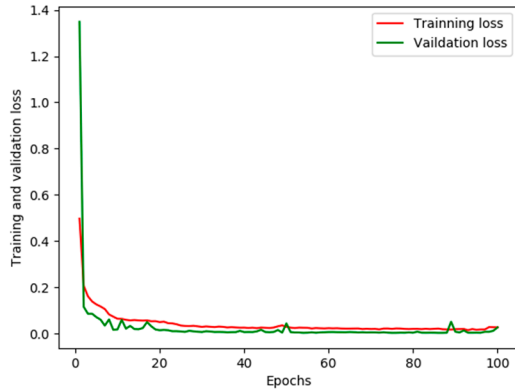
1st time: Training and validation with dropout = 0.3



2nd time: Training and validation with dropout = 0.4

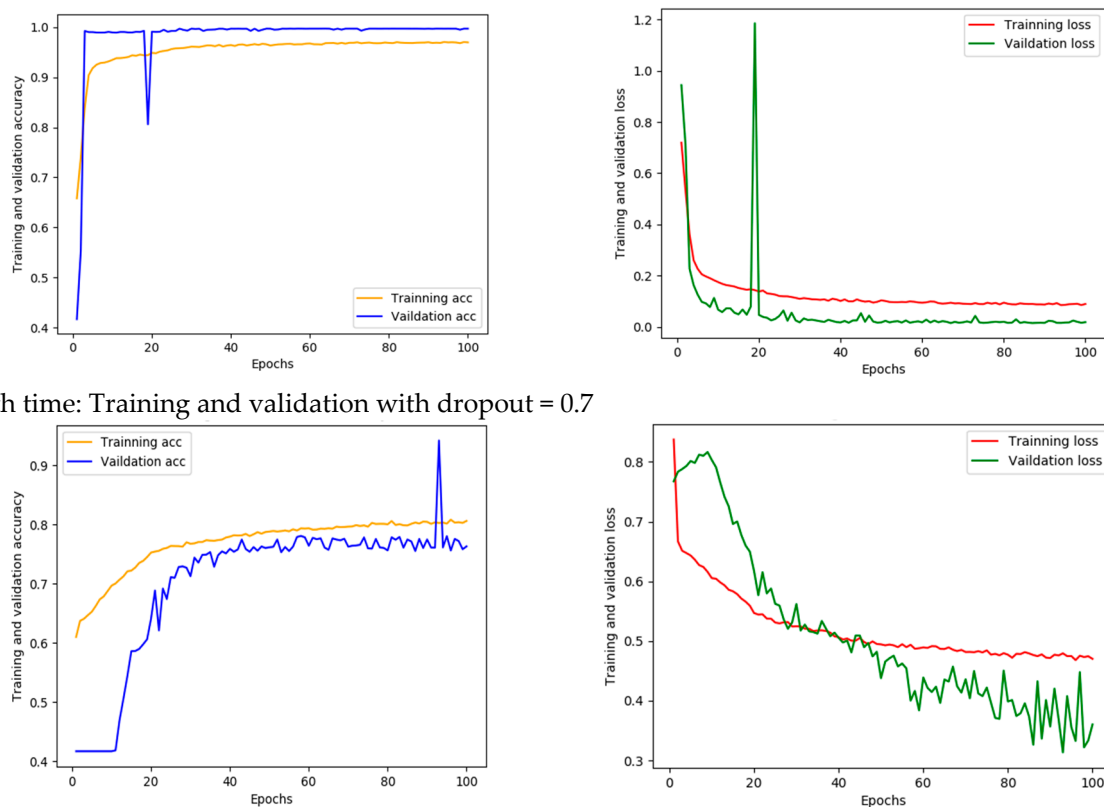


3rd time: Training and validation with dropout = 0.55



4th time: Training and validation with dropout = 0.6

Figure 13. Cont.



5th time: Training and validation with dropout = 0.7

6th time: Training and validation with dropout = 0.9

Figure 13. Trends in training and validation losses with dropout added.

During the training process, the loss values of the model without the dropout technique were smaller than those of the model with the dropout technique. As can be seen from Figure 12, the variation of the values of validation loss and accuracy in the model without the dropout technique is very large. On the contrary, the variation of validation loss and accuracy in the model with the dropout technique is relatively small. Its curve tends to be smooth. When the model is not trained with the dropout technique, the training data are overfitted in order to minimize the loss function. This leads to minimizing the loss of the model. On the other hand, the accuracy of the model with the dropout technique is high. This is because when the parameters are updated, some parameters are randomly selected to be updated in order to avoid the learning record of the inherent model parameters. When testing the model incorporating the dropout technique, the learning parameters of each small network are fully considered. So, the problem of overfitting to individual parameters does not occur. This greatly reduces the overfitting of the model. The above results demonstrate that adding dropout technique will greatly reduce the overfitting and make it perform better in the test dataset.

As shown by Figure 13, the groups 3 and the 4 are a little better than groups 1, 2, 5 and 6. The curve is smoothest and best when dropout = 0.55. In Figures 12 and 13, the x-axis represents the epochs of training and validation. The y-axis represents the accuracy or loss rate of training and validation. The effect of dropout on accuracy and loss is shown in Figure 14.

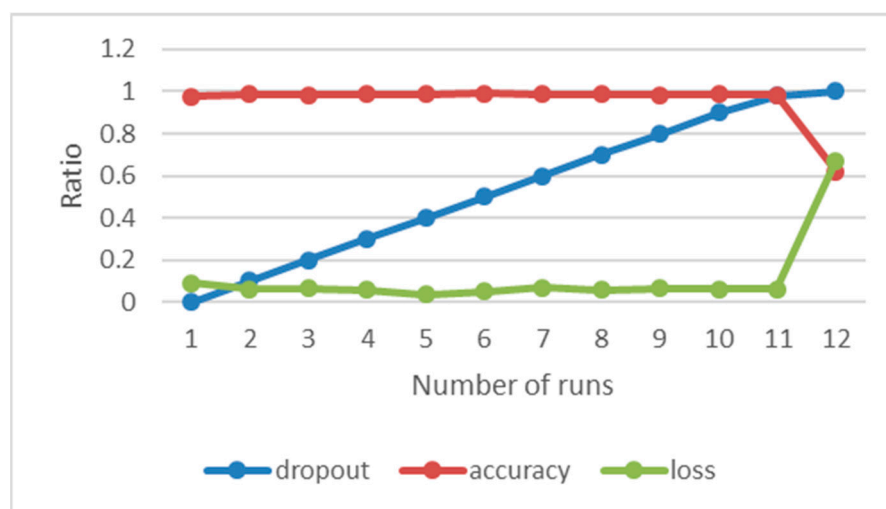


Figure 14. Dropout effect on accuracy and loss.

For the CICIDS2017 dataset, the number and type distribution of streams are extracted using the CNN-SDN anomaly detection algorithm. As shown in Table 4, the proportion of benign flows, port scans, and Heart Bleed attack flows is much larger than others. This algorithm is able to detect the types of anomalous flows that cannot be detected by traditional algorithms.

Table 4. CICIDS2017 Dataset Anomaly Distribution.

Flow Type	Number	Percentage
Benign	305,429	55.14652%
BotNet	11,424	2.06265%
Brute Force	1427	0.25765%
DDOS	12,420	2.24248%
Goldeneye	6864	1.23932%
Dos Hulk	10,345	1.86783%
Dos Slow http	3819	0.68954%
Dos slow loris	4258	0.76880%
FTP Patator	3826	0.69080%
Port Scan	108,140	19.52514%
Heart Bleed	76,368	13.78857%
SQL Injection	47	0.00849%
SSH Patator	2135	0.38548%
Infiltration	5364	0.96849%
Web Attack	1243	0.22443%
XSS	741	0.13379%

6.4. Comparison of Algorithms

The CNN-SDN model was analyzed based on the original features extracted from the traffic shown in Figure 15. As seen from the results, the CNN-SDN model performs better than the traditional machine learning algorithms KNN, RF and NB models [37], better than Zhang's single model of CNN11, CNN2 and LSTM2, and close to the hybrid model of CNN + LSTM2 [14,40].

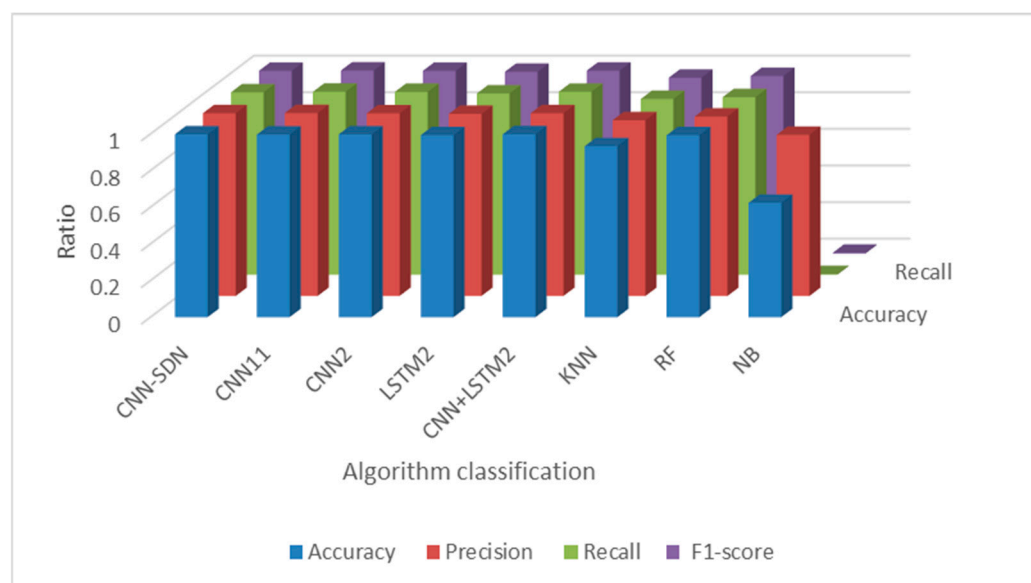


Figure 15. CICIDS2017 dataset experiment results.

The proposed model has good efficiency in detecting anomalous traffic in four metrics: accuracy, precision, recall, and F1-score. The performance is only slightly improved compared with others. However, in a real network environment, it is desirable to detect as many traffic packets with anomalous behavior as possible due to the very large volume of traffic data. Our model can further optimize the parameters to improve the classification metrics.

Multi-classification experiments were conducted on the CICIDS2017 dataset, as shown in Figure 16. It was found that the traffic classification accuracy on the CICIDS2017 dataset is more than 99.88%. The statistical method and traditional machine learning algorithms cannot represent the complete flow information. These methods may lead to low classification accuracy. While the network structure proposed by Zhang et al. [14] is partially similar to the proposed model, our model obtains better performance with fewer parameters and converges faster, and our model can be applied in SDN for real-time online anomaly detection and mitigation. In terms of convergence and accuracy, the proposed model performs excellently.

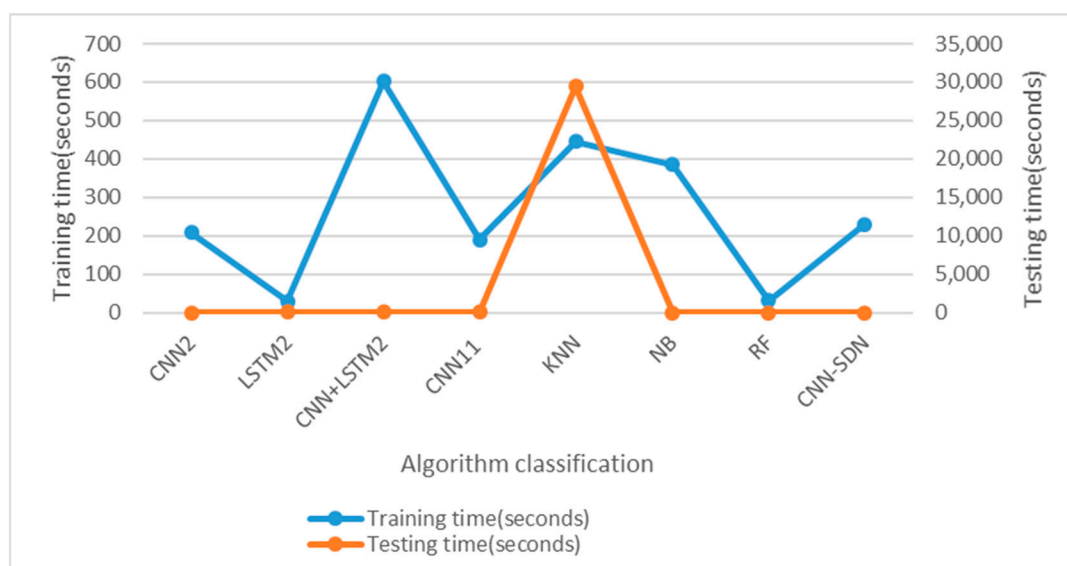


Figure 16. Analysis of model convergence.

The anomaly detection algorithm detects suspicious traffic and identifies it as DDOS or port scanning or Heart Bleed infiltration. The mitigation module can use three mitigation strategies to handle the anomaly accordingly [5]. The anomaly mitigation module is highly dependent on the attack strength. Here we investigate whether mitigation is used to reduce network bandwidth usage and improve system throughput to measure system operation. These results are shown in Figures 17 and 18. There is a corresponding reduction in bandwidth usage after applying mitigation strategies. Compared to other algorithms, the bandwidth usage of our model is reduced by 8.9% on average and throughput is improved by 10% on average.

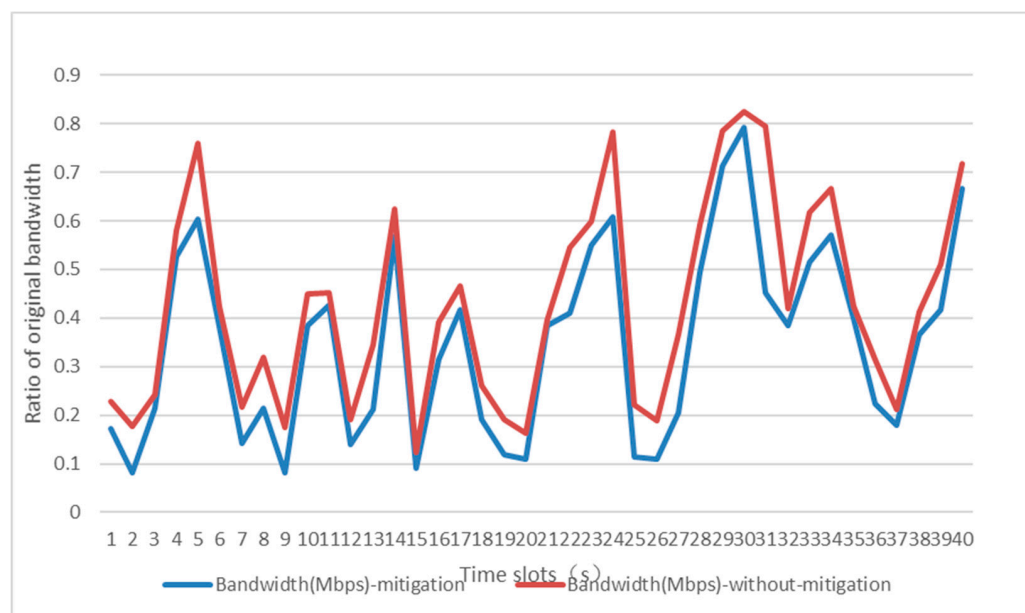


Figure 17. Comparison of bandwidth with and without mitigation strategies.

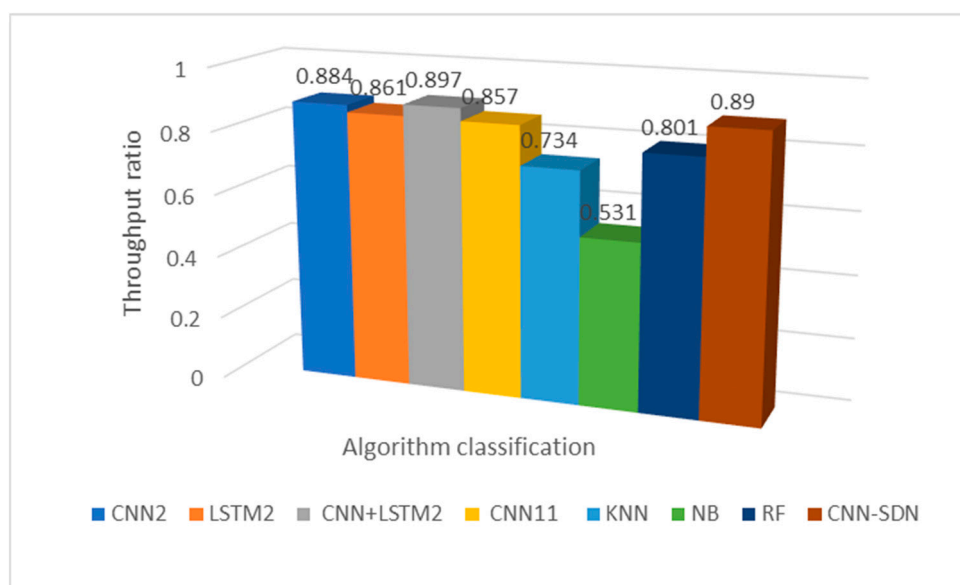


Figure 18. Comparison of throughput of each algorithm.

7. Conclusions

The artificial design and extraction of traffic features can lose some traffic information in network anomaly detection, which affects the accuracy of the detection. In this work,

the raw information of traffic is extracted. Then, the spatial and temporal features learned from the raw network traffic of CNN models are used to detect traffic anomalies. The mitigation strategy with blocking, discarding, and redirecting traffic significantly reduces the harm of anomalies. The proposed method is validated using Mininet and RYU to show the capability of the proposed method in improving network performance. In the future, the model can be enabled to detect more unknown types of attacks without training and also to identify high- and low-rate data stream anomalies by analyzing novel anomaly detection techniques using other machine learning methods. This will be attempted for anomaly traffic detection in CDN to improve the performance of CDN.

Author Contributions: Conceptualization, H.L.; methodology, H.L.; software, H.L.; validation, H.L. and H.W.; formal analysis, H.W.; investigation, H.L.; resources, H.L.; data curation, H.L.; writing—original draft preparation, H.L.; writing—review and editing, H.W.; visualization, H.L.; supervision, H.W.; project administration, H.L.; funding acquisition, H.L. and H.W. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Shanghai Key Science and Technology Project (19DZ1208903); National Natural Science Foundation of China (Grant Nos. 61572325); Shanghai Key Science and Technology Project in Information Technology Field (Grant Nos. 14511107902 and 16DZ1203603); Shanghai Leading Academic Discipline Project (No. XTKX2012); Shanghai Engineering Research Center Project (Nos. GCZX14014 and C14001); Introduction and Cultivation Program for Young Innovative Talents of Universities in Shandong (2021QCY003).

Data Availability Statement: Data are available in a publicly accessible repository that does not issue DOIs. Publicly available datasets were analyzed in this study.

Acknowledgments: The authors would like to thank the editors and anonymous reviewers for their valuable comments and suggestions. The authors would like to acknowledge the support from the Flow Computing Laboratory at University of Shanghai for Science and Technology.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Wazid, M.; Reshma Dsouza, P.; Das, A.K.; Bhat, K.V.; Kumar, N.; Rodrigues, J.J. RAD-EI: A routing attack detection scheme for edge-based Internet of Things environment. *Int. J. Commun. Syst.* **2019**, *32*, e4024. [\[CrossRef\]](#)
2. Ramprasath, J.; Seethalakshmi, V. Secure access of resources in software defined networks using dynamic access control list. *Int. J. Commun. Syst.* **2021**, *34*, e4607.
3. Yu, W.; Liang, F.; He, X.; Hatcher, W.G.; Lu, C.; Lin, J.; Yang, X. A Survey on the Edge Computing for the Internet of Things. *IEEE Access* **2017**, *6*, 6900–6919. [\[CrossRef\]](#)
4. Jararweh, Y.; Doulat, A.; Darabseh, A.; Alsmirat, M.; Al-Ayyoub, M.; Benkhelifa, E. SDMEC: Software Defined System for Mobile Edge Computing. In Proceedings of the IEEE International Conference on Cloud Engineering Workshop, Berlin, Germany, 4–8 April 2016.
5. Ramprasath, J.; Seethalakshmi, V. Improved Network Monitoring Using Software-Defined Networking for DDoS Detection and Mitigation Evaluation. *Wirel. Pers. Commun.* **2021**, *116*, 2743–2757. [\[CrossRef\]](#)
6. Ramprakash, P.; Sakthivadivel, M.; Krishnaraj, N.; Ram-prasath, J. Host-based intrusion detection system using sequence of system calls. *Int. J. Eng. Manag. Res. (IJEMR)* **2014**, *4*, 241–247.
7. Carl, G.; Kesidis, G.; Brooks, R.; Rai, S. Denial-of-service attack-detection techniques. *IEEE Internet Comput.* **2006**, *10*, 82–89. [\[CrossRef\]](#)
8. Shakya, S. Process Mining Error Detection for Securing the IoT System. *J. ISMAC* **2020**, *2*, 147–153. [\[CrossRef\]](#)
9. Zhou, H.; Wang, Y.; Lei, X.; Liu, Y. A method of improved CNN traffic classification. In Proceedings of the 2017 13th International Conference on Computational Intelligence and Security (CIS), Hong Kong, China, 15–18 December 2017; pp. 177–181.
10. Kim, J.; Kim, J.; Thu, H.L.T.; Kim, H. Long short term memory recurrent neural network classifier for intrusion detection. In Proceedings of the 2016 International Conference on Platform Technology and Service (PlatCon), Jeju, Republic of Korea, 15–17 February 2016; pp. 1–5.
11. Shukla, P.K.; Maheshwary, P.; Subramanian, E.; Shilpa, V.J.; Varma, P.R.K. Traffic flow monitoring in software-defined network using modified recursive learning. *Phys. Commun.* **2023**, *57*, 101997. [\[CrossRef\]](#)
12. Zavrak, S.; Iskefiyeli, M. Flow-based intrusion detection on software-defined networks: A multivariate time series anomaly detection approach. *Neural Comput. Appl.* **2023**, *35*, 12175–12193. [\[CrossRef\]](#)
13. Tang, D.; Wang, X.; Li, X.; Vijayakumar, P.; Kumar, N. AKN-FGD: Adaptive Kohonen Network based Fine-grained Detection of LDoS Attack. *IEEE Trans. Dependable Secur. Comput.* **2023**, *20*, 273–287. [\[CrossRef\]](#)

14. Zhang, Y.; Chen, X.; Jin, L.; Wang, X.; Guo, D. Network intrusion detection: Based on deep hierarchical network and original flow data. *IEEE Access* **2019**, *7*, 37004–37016. [[CrossRef](#)]
15. Lopez-Martin, M.; Carro, B.; Sanchez-Esguevillas, A.; Lloret, J. Conditional Variational Autoencoder for Prediction and Feature Recovery Applied to Intrusion Detection in IoT. *Sensors* **2017**, *17*, 1967. [[CrossRef](#)] [[PubMed](#)]
16. Edge Computing Industry Alliance. *Edge Computing Reference Architecture 2.0*; Alliance of Industrial Internet (ALL): Beijing, China, 2017.
17. Ali, T.E.; Chong, Y.-W.; Manickam, S. Comparison of ML/DL Approaches for Detecting DDoS Attacks in SDN. *Appl. Sci.* **2023**, *13*, 3033. [[CrossRef](#)]
18. Vaquero, L.M.; Roderio-Merino, L. Finding your way in the fog: Towards a comprehensive definition of fog computing. *ACM SIGCOMM Comput. Commun. Rev.* **2014**, *44*, 27–32. [[CrossRef](#)]
19. McKeown, N.; Anderson, T.; Balakrishnan, H.; Peterson, L.; Rexford, J.; Shenker, S.; Turner, J. Openflow: Enabling innovation in campus networks. *ACM SIGCOMM Comput. Commun. Rev.* **2008**, *38*, 69–74. [[CrossRef](#)]
20. Lin, T.; Park, B.; Bannazadeh, H.; Leon-Garcia, A. Demo abstract: End-to-end orchestration across sdi smart edges. In Proceedings of the 2016 IEEE/ACM Symposium on Edge Computing (SEC), Washington, DC, USA, 27–28 October 2016; pp. 127–128.
21. Ren, G.; Zhang, Y.; Zhang, S.; Long, H. EdgeDDoS Attack Detection Method Based on Software Defined Networks. In Proceedings of the ICA3PP, Virtual Event, 3–5 December 2021; pp. 597–611.
22. Zhou, H.; Zheng, Y.; Jia, X.; Shu, J. Collaborative prediction and detection of DDoS attacks in edge computing: A deep learning-based approach with distributed SDN. *Comput. Netw.* **2023**, *225*, 109642. [[CrossRef](#)]
23. Yang, Y.; Wang, J.; Zhai, B.; Liu, J. IoT-Based DDoS Attack Detection and Mitigation Using the Edge of SDN. In Proceedings of the Cyberspace Safety and Security: 11th International Symposium, CSS 2019, Guangzhou, China, 1–3 December 2019.
24. Ilango, H.S.; Ma, M.; Su, R. A FeedForward–Convolutional Neural Network to Detect Low-Rate DoS in IoT. *Eng. Appl. Artif. Intell.* **2022**, *114*, 105059. [[CrossRef](#)]
25. Garg, S.; Kaur, K.; Kumar, N.; Rodrigues, J.J.P.C. Hybrid Deep-Learning-Based Anomaly Detection Scheme for Suspicious Flow Detection in SDN: A Social Multimedia Perspective. *IEEE Trans. Multimed.* **2019**, *21*, 566–578. [[CrossRef](#)]
26. Nunes, B.A.A.; Mendonca, M.; Nguyen, X.-N.; Obraczka, K.; Turletti, T. A survey of software defined networking: Past, present, and future of programmable networks. *IEEE Commun. Surv. Tutor.* **2014**, *16*, 1617–1634. [[CrossRef](#)]
27. Carvalho, L.F.; Abrão, T.; de Souza Mendes, L.; Proença, M.L., Jr. An ecosystem for anomaly detection and mitigation in software defined networking. *Expert Syst. Appl.* **2018**, *104*, 121–133. [[CrossRef](#)]
28. Matsugu, M.; Mori, K.; Mitari, Y.; Kaneda, Y. Subject independent facial expression recognition with robust face detection using a convolutional neural network. *Neural Netw.* **2003**, *16*, 555–559. [[CrossRef](#)] [[PubMed](#)]
29. Wang, H. On Anomaly Detection and Defense Resource Allocation of Industrial Control Networks. Master's Thesis, Zhejiang University, Hangzhou, China, 2014.
30. Zhang, Y.; Li, X.; Li, D.; Yang, H. Abnormal flow monitoring of industrial control network based on convolutional neural network. *J. Comput. Appl.* **2019**, *39*, 1512.
31. Zhong, J.; Yang, B. Review on the research progress of convolutional neural networks. *J. South China Univ. (Nat. Sci. Ed.)* **2016**, *30*, 66–72.
32. Liang, C.; Xiaoming, D.; Mingquan, Z.; Zhongke, W.; Ye, Y.; Shuo, Y.; Hongan, W. Convolution neural network in image comprehension. *Acta Autom. Sin.* **2016**, *9*, 1300–1312.
33. Garg, S.; Kaur, K.; Kumar, N.; Kaddoum, G.; Zomaya, A.Y.; Ranjan, R. A Hybrid Deep Learning-Based Model for Anomaly Detection in Cloud Datacenter Networks. *IEEE Trans. Netw. Serv. Manag.* **2019**, *16*, 924–935. [[CrossRef](#)]
34. Li, L.; Ying, S. Softmax layer implementation of neural network based on FPGA. *Mod. Comput.* **2017**, *26*, 21–24.
35. Anderson, J.P. *Computer Security Threat Monitoring and Surveillance*; Technical Report; James P. Anderson Company: Morganville, NJ, USA, 26 February 1980.
36. Weller-Fahy, D.J.; Borghetti, B.J.; Sodemann, A.A. A Survey of Distance and Similarity Measures Used Within Network Intrusion Anomaly Detection. *IEEE Commun. Surv. Tutor.* **2014**, *17*, 70–91. [[CrossRef](#)]
37. Sharafaldin, I.; Lashkari, A.H.; Ghorbani, A.A. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In Proceedings of the International Conference on Information Systems Security & Privacy, Funchal, Portugal, 22–24 January 2018.
38. Maseer, Z.K.; Yusof, R.; Bahaman, N.; Mostafa, S.A.; Foozy, C.F. Benchmarking of Machine Learning for Anomaly Based Intrusion Detection Systems in the CICIDS2017 Dataset. *IEEE Access* **2021**, *9*, 22351–22370. [[CrossRef](#)]
39. Van Der Maaten, L. Python Implementation of t-SNE. Available online: <https://lvdmaaten.github.io/tsne> (accessed on 1 January 2023).
40. Cite Github. Available online: <https://github.com/chenxu93/abnormal-traffic> (accessed on 25 July 2022).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.